

■# Futures Terminal - Complete Feature Reference

Generated on: 2026-04-15

1) What This Software Is Doing

This is a desktop-first futures trading terminal focused on real-time market intelligence and paper-execution workflow.

The system continuously does four jobs:

1. Collects market data from Binance and additional Top-20 exchange adapters (where live/available).
2. Reconstructs and merges order-book/trade context into one terminal state.
3. Builds heatmap + ladder + candle visualization for fast decision support.
4. Exposes trading controls and safety controls in paper mode, while recording all raw data for replay.

Execution mode currently exposed in snapshot payload: paper.

2) High-Level Architecture

Frontend (React + TypeScript)

- Live terminal layout with chart, ladder, dominance panel, signal blocks, and controls.
- Pair search/switch UI with exchange-coverage modal.
- WebSocket + REST bootstrap client for low-latency updates.

Backend (FastAPI + async services)

- Public Binance stream ingestion (depth, trades, ticker, mark/funding, kline, liquidation).
- Top-20 exchange polling layer via CCXT (feature/adapter availability depends on venue and network conditions).
- Local order-book sync, merged book/trade/mark context, heatmap/ladder frame generation.
- REST + WebSocket distribution to UI.
- Raw event recording to SQLite + JSONL replay files.

Storage

- SQLite table `raw\_events` for timestamped event persistence.
- Rolling replay files (`.jsonl`) per session.

3) Top Navigation Features

A) Pair Management

- Active pair display.
- Search input with live suggestion dropdown.
- Switch Pair action with validation for `BASEUSDT` format.

- Supports selecting from full exchange symbol list fetched from backend.

B) Exchange-Coverage Info (i button)

Opens a modal showing:

- Total exchange catalog count (20).
- Supported count for active pair.
- Live connected count.
- Per-feature contributor counts:
 - book/spread
 - trades
 - candles
 - mark/funding
 - liquidations
- Symbol resolution details (requested/normalized).
- Per-exchange status rows (Live / Ready / Pair not supported / API blocked-temporary error / Adapter unavailable etc.).

C) Route Tabs

- Terminal
- Replay
- Settings
- Logs

4) Main Terminal Layout

Left Sidebar

DOM / Order Book card

- Best bid
- Best ask
- Spread
- Total depth update count
- Current level counts (bids/asks)

Order Entry card

- Quantity input
- Buy Market button
- Sell Market button

Positions card

- Placeholder state: no open positions

Open Orders card

- Placeholder state: no open orders

Center Area

A) Unified Candlestick + Heatmap Chart

This is a single coordinated chart engine with shared X/Y mapping for candles, heatmap rows, crosshair and price axis.

Toolbar controls:

- Low threshold slider (heatmap color lower clamp)
- High threshold slider (heatmap color upper clamp)
- Row Size slider (`1x` to `50x`)
- Heatmap Range slider (`1.00x` to `50.00x`)
- Timeframe buttons: `1m`, `3m`, `5m`, `15m`, `30m`, `1h`, `2h`, `4h`
- Full Screen toggle
- Reset Thresholds
- Reset View
- Hide/Show Heatmap

Core behavior:

- Candles and heatmap rendered in same coordinate system.
- Right-side future gap + price axis region.
- Current price dashed line + right-axis tag.
- Crosshair with vertical/horizontal guides.
- Compact OHLC tooltip box on hover.

Mouse/trackpad interaction:

- Drag in plot pans chart horizontally.
- Wheel in plot zooms time scale around cursor anchor.
- Horizontal two-finger movement pans chart.
- Wheel on price-axis side performs vertical scale zoom.
- Double-click plot resets horizontal view (follow-live style reset).
- Double-click price axis resets Y auto-fit.

Fullscreen mode:

- Keeps chart controls available.

- Shows on-chart Liquidity Area probe table in lower-left.

B) Dominance Panel (between chart and ladder)

Real-time buy vs sell dominance computed from live ladder rows.

Displayed values:

- Sell % and Buy %
- Vertical split meter (red sell, green buy)
- Sell liquidity amount
- Buy liquidity amount
- Anchor price used
- Dominance label (`Buy Dominant`, `Sell Dominant`, `Balanced`)
- Status string

Settings (Cfg button):

- Anchor mode: Current or Custom
- Custom anchor price input
- Rows Above (1..100)
- Rows Below (1..100)

Computation logic:

- Anchor row selected from current price or nearest row to custom anchor.
- Sums liquidity above anchor for sell side and below anchor for buy side.
- Percentages are normalized from total selected liquidity.

C) Live Ladder Panel

- Displays ladder rows with price + colored liquidity boxes.
- Side coloring:
 - above current: red-tinted
 - below current: green-tinted
 - at current: neutral/current highlight
- Auto-centers around current row when user is idle.
- User manual scrolling respected before recentering.

5) Bottom Analytics Wrappers

CVD

- Proxy state (buy/sell/neutral pressure)
- Trade speed (10s)

Liquidity Area (Probe Table)

- 3x3 liquidity matrix around pointer location on chart:
- rows: top/mid/bottom (relative to pointer row)
- cols: left/mid/right (relative to pointer column)
- Additional price column per row.
- Designed to inspect local liquidity neighborhood under cursor.

Market Snapshot

- Bid / Spread / Ask
- Trade/s / Mark / Funding
- Book sync state / Last trade / Liquidations in 60s

Bias Meter

- Long score
- Short score
- Confidence

6) Secondary Analytics + Safety

Positioning Context

- Streams active count
- Liquidations (60s)

Spoofing

- Sync anomaly count
- Depth health (Stable / Resyncing)

Absorption

- Proxy score
- Spread

Safety

- Emergency Close button

Status

- Event count
- Book synced (YES/NO)
- Depth update ID
- Sync failures

7) Replay / Settings / Logs Pages

Replay page

- Reads replay file list from backend (`/replay/files`).
- Shows available `.jsonl` session files.

Settings page

- Informs user to edit JSON config files under `config/`.

Logs page

- Points to backend log path (`backend/data/logs/terminal.log`).

8) Backend Data Pipeline Details

A) Binance Real-Time Stream Ingestion

Connected streams include:

- depth
- aggTrade
- bookTicker
- markPrice
- kline_1m
- forceOrder

Depth sync behavior:

- Buffers depth events before sync.
- Attempts snapshot resync through REST depth endpoint.
- Applies forced snapshot sync fallback to avoid endless stale loops.
- Updates sync-failure counters and health flags.

B) Top-20 Exchange Aggregation Layer

The system maintains a 20-exchange catalog and tracks:

- support for active pair
- connection freshness
- per-feature activity flags

Poller features (adapter/venue dependent):

- order book polling
- trades polling

- ohlcv polling
- mark/funding polling
- liquidation polling

Adapter limitations are surfaced to UI via reason/status fields.

C) Merged Market State

Live state merges:

- Binance local book
- external books from supported exchanges
- external trade feeds
- external OHLCV feeds
- external mark/funding feeds
- external liquidation feeds

Derived values:

- best bid/ask and spread
- trade rate over last 10s
- liquidations over last 60s
- stream/event counters

9) Heatmap + Ladder Engine (Backend)

Price Bucketizer

- Uses ``bucket_size = tick_size * ticks_per_bucket``.
- Maps prices to row indices and back.

Heatmap Frame Builder

- Builds one heatmap column per emit cycle.
- Aggregates bid + ask liquidity into row buckets around center price window.

Heatmap Store

- Ring buffer style storage of columns.
- Max columns default: 14,400 (~2h at 500ms cadence).
- Supports bootstrap retrieval of recent columns.

Ladder Snapshot Builder

- Builds fixed window around center row.

- Produces rows with:
- price
- liquidity
- side (`above`, `below`, `at`)

Emit cadence

- Snapshot and coverage updates emitted on interval.
- Heatmap/ladder messages emitted at heatmap bucket cadence when data is ready.

10) API + WebSocket Contracts

REST endpoints

- `GET /health`
- `GET /settings`
- `GET /settings/symbols`
- `POST /settings/symbol`
- `GET /terminal/snapshot`
- `GET /terminal/heatmap/bootstrap`
- `GET /terminal/source-coverage`
- `POST /trading/test-order`
- `GET /positions`
- `GET /replay/files`

WebSocket endpoint

- `ws://127.0.0.1:8000/ws/terminal`

Message types consumed by frontend store

- `market\_event`
- `terminal\_snapshot`
- `symbol\_changed`
- `heatmap\_frame`
- `ladder\_snapshot`
- `source\_coverage\_update`
- `symbol\_resolution`

11) Data Recording and Replay

Raw recorder behavior:

- Every incoming event is queued.
- Writes to SQLite `raw\_events` table.
- Writes JSON lines to session replay file.
- Handles queue overflow with drop warning logging.

12) Current Trading Mode and Constraints

- Execution mode exposed as paper.
- Test-order route currently scaffolded (no direct live order routing enabled here).
- Position/order cards are currently UI placeholders.
- Multi-exchange availability depends on adapter support + venue/network/API access from your machine.

13) Practical Usage Flow

1. Start backend + frontend (or launcher batch).
2. Select pair in top search box and click Switch Pair.
3. Open i coverage panel to verify how many exchanges are contributing live.
4. Use chart + ladder + dominance + liquidity probe for decision context.
5. Use paper trade controls and safety button while monitoring status and sync health.
6. Review replay files for post-session analysis.

14) What Is Implemented vs Placeholder

Implemented live:

- Real-time chart with heatmap and interaction engine
- Ladder and dominance
- Pair switching + coverage telemetry
- Snapshot/status analytics
- Backend stream ingestion, merging, and recording

Currently placeholder/scaffold:

- Full live execution routing in UI order buttons
- Non-empty positions/open-orders data source integration
- Fully uniform live support across all top-20 exchanges under all network conditions

If you want, the next revision of this document can include architecture diagrams and per-feature data formulas (for each proxy metric) on separate pages.